XML

SYSTEM INTEGRATION

Arturo Mora-Rioja
amri@ek.dk

EK

XML

- eXtensible Markup Language
- Metalanguage. It defines the syntax for XML-based languages

```xml
<?xml version="1.0"?>
<employee id="45">
     <first_name>Lars G.</first_name>
     <last_name>Hansen</last_name>
     <professional_information>
          <department>IT</department>
          <specialisation>Cybersecurity</specialisation>
          <office>Sydhavn</office>
     </professional_information>
</employee>
```

XML file

- Validation:
  - Validate XML
  - Code Beautify XML Validator
  - Free Formatter XML Validator

EK

XML Use Cases

- Data interchange and APIs
  - SOAP web services, financial messaging, B2B integration, enterprise integration (ESB)
- Configuration and metadata
  - Java and Apache ecosystems, build tools (e.g., Maven, Ant), game and device configuration
- Office and document formats, publishing and typesetting workflows
  - Microsoft Office, OpenDocument (e.g., LibreOffice), EPUB ebooks
- Web content and graphics
  - RSS and Atom feeds, SVG, MathML, XHTML
- Domain-specific business standards
  - Finance, tax and regulatory, healthcare, telecom, geospatial
- CMSs

Well-Formed XML

- An XML document must be well-formed
  - There must be only one root element
  - All tags must be closed
    - Wrong
    - Right

```
<recipe>A Spanish omelette has <ingredient>eggs and <ingredient>potatoes</ingredient></recipe>
```

```
<recipe>A Spanish omelette has <ingredient>eggs</ingredient> and <ingredient>potatoes</ingredient></recipe>
```

  - Tag overlapping is forbidden
    - Wrong
    - Right

```
<book><title>The Mezzanine</title> by <author>Nicholson Baker</book></author>
```

```
<book><title>The Mezzanine</title> by <author>Nicholson Baker</author></book>
```

  - Attribute values must be between quotes
    - Wrong
    - Right

```
<employee id=45>Gunnar J. Jensen</employee>
```

```
<employee id="45">Gunnar J. Jensen</employee>
```

  - The characters < and > must be represented through character entities
    - Wrong
    - Right

```
<formula>x > y</formula>
```

```
<formula>x &gt; y</formula>
```

XML Namespaces

- When combining XML files, duplicated tag names may cause conflict
  - E.g., different departments in a company use the same name for their tags. When merging the information for a specific customer, it looks like this:

```
<customer>
    <address>Guldbergsgade 29N, 2200 København N</address>
    <address>ek@ek.dk</address>
    <address>94.145.192.43</address>
</customer>
```

  - XML's namespaces extension provides a solution

```
<?xml version="1.0" encoding="iso-8859-1"?>
<customers
    xmlns:service="https://mycompany.com/service"
    xmlns:marketing="https://mycompany.com/marketing"
    xmlns:it="https://mycompany.com/it">
    <customer>
        <service:address>Guldbergsgade 29N, 2200 København N</service:address>
        <marketing:address>ek@ek.dk</marketing:address>
        <it:address>94.145.192.43</it:address>
    </customer>
</customers>
```

XML Libraries

- Streaming writers: they allow to create XML content manually

Language	Standard Library	Third-Party
Java	StAX (XMLStreamWriter) SAX (ContentHandler) DOM	Woodstox
Python	xml.sax.saxutils.XMLGenerator xml.etree.ElementTree (manual building) xml.dom.minidom (DOM writer)	lxml.etree.xmlfile
JavaScript / Node.js	XMLSerializer (browser)	xmlbuilder2 fast-xml-parser node-xml-stream
C# (.NET)	System.Xml.XmlWriter System.Xml.XmlTextWriter	
PHP	XMLWriter DOMDocument SimpleXMLElement	

XML Schema Definition

- Besides well-formedness, XML documents must be validated against their schema definition
- Standards
  - DTD (Document Type Definition)
    - Not written in XML
    - Does not support data types
    - Does not support namespaces
    - Almost not in use nowadays
  - XSD (XML Schema Definition)
    - Written in XML
    - Supports strong typing, namespaces, and cardinality

```
<!-- Sample DTD -->
<!ELEMENT Recipe (Name, Description?, Ingredients?, Instructions?)>
<!ELEMENT Name (#PCDATA)>
<!ELEMENT Description (#PCDATA)>
<!ELEMENT Ingredients (Ingredient*)>
<!ELEMENT Ingredient (Amount, Item)>
<!ELEMENT Amount (#PCDATA)>
<!ATTLIST Amount unit CDATA #REQUIRED>
<!ELEMENT Item (#PCDATA)>
<!ATTLIST Item optional CDATA "0" vegetarian CDATA "yes">
<!ELEMENT Instructions (Step+)>
<!ELEMENT Step (#PCDATA)>
```

XSD – XMLSchema

- XSD documents use the `xs` namespace
- The root element is `<xs:schema>`. Attributes:
  - `xmlns:xs` references the namespace that defines the tags and attributes used by XSD
  - `targetNamespace` references the namespace where the tags are defined
  - `xmlns` references the default namespace to use for tags defined without a prefix
  - `elementFormDefault="qualified"` makes the said assignment implicit

```
<?xml version="1.0"?>
    <xs:schema
        xmlns:xs="http://www.w3.org/2001/XMLSchema"
        targetNamespace="http://www.company.com"
        xmlns="http://www.company.com"
        elementFormDefault="qualified">
    ...
</xs:schema>
```

XSD – XMLSchema

- Data types: `xs:string, xs:decimal, xs:integer, xs:boolean, xs:date, xs:time`
- Elements
  - Simple. It contains only text

```
<xs:element name="Age" type="xs:integer"/>
```

```
<age>34</age>
```

    - There can be a default value and even an immutable value

```
<xs:element name="language" type="xs:string" default="da"/>
<xs:element name="country" type="xs:string" fixed="dk"/>
```

  - Complex. It can contain attributes and/or other elements
    - Attributes. Defined with a data type

```
<xs:attribute name="city" type="xs:string"/>
```

```
<hotel city="Berlin">
  Ritz
</hotel>
```

      - There can be a default value, an immutable value, and whether a value is mandatory

```
<xs:attribute name="status" type="xs:string"
    default="active" use="required"/>
```

XSD – XMLSchema

- Elements (cont.)
  - Complex (cont.)
    - Elements that contain elements

```
<student>
  <first-name>Linus</first-name>
  <last-name>Torvalds</last-name>
</student>
```

```
<xs:element name="student">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="first-name" type="xs:string"/>
            <xs:element name="last-name" type="xs:string"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

      - The complex type can be stored and reused for further elements

```
<xs:complexType name="person-info">
...
<xs:element name="teacher" type="person-info"/>
```

XSD – XMLSchema

- Elements (cont.)
  - Complex (cont.)
    - Empty elements. They do not use a closing tag

```
<xs:element name="product">
    <xs:complexType>
        <xs:complexContent>
            <xs:restriction base="xs:integer">
                <xs:attribute name="id" type="xs:positiveInteger"/>
            </xs:restriction>
        </xs:complexContent>
    </xs:complexType>
</xs:element>
```

```
<product id="1345"/>
```

XSD – XMLSchema

- Elements (cont.)
  - Complex (cont.)
    - Elements that contain both text and elements

```
<xs:element name="expedition">
    <xs:complexType mixed="true">
        <xs:sequence>
            <xs:element name="key-date" type="xs:date"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

```
<expedition>
    We arrived to the top of the mountain
    at <key-date>2026-01-24</key-date> in the evening.
</expedition>
```

XSD – XMLSchema

- Constraints
  - Content-based

```
<xs:element name="salary">
    <xs:simpleType>
        <xs:restriction base="xs:decimal">
            <xs:minInclusive value="30000"/>
            <xs:maxInclusive value="120000"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

  - Length-based

```
<xs:element name="password">
    <xs:simpleType>
        <xs:restriction base="xs:string">
            <xs:length value="8"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

XSD – XMLSchema

- Constraints (cont.)
  - Value-based
    - Enumeration

```
<xs:element name="wine">
    <xs:simpleType>
        <xs:restriction base="xs:string">
            <xs:enumeration value="Red"/>
            <xs:enumeration value="White"/>
            <xs:enumeration value="Rosé"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

      - The type can be stored for further elements

```
<xs:simpleType name="wineType">
```

XSD – XMLSchema

- Constraints (cont.)
  - Value-based (cont.)
    - Pattern. Uses regEx

```
<xs:element name="letters">
    <xs:simpleType>
        <xs:restriction base="xs:string">
            <xs:pattern value="([a-z])*"/>
        </xs:restriction>
    </xs:simpleType>
</xs:element>
```

  - Other constraints
    - totalDigits, fractionDigits
    - maxExclusive, maxInclusive, minExclusive, minInclusive
    - maxLength, minLength
    - whiteSpace

XSD – XMLSchema

- Complex type indicators
  - `<xs:all>` All child elements must appear
  - `<xs:choice>` Only one child element can appear
  - `<xs:sequence>` Child elements must appear in a specific order
  - `maxOccurs, minOccurs` Delimit the range of possible occurrences of an element
  - `<xs:group>` Group together elements
  - `<xs:attributeGroup>` Group together attributes

- Free online XML Schema Validator

XML Object Libraries

- Object-to-XML binders: they allow to create XML content from models

Language	Standard Library	Third-Party
Java	JAXB (javax.xml.bind/jakarta.xml.bind)	XStream Simple XML Framework Jackson XML module
Python		xsdata generateDS dicttoxml
JavaScript / Node.js		js2xmlparser xml2js
C# (.NET)	XmlSerializer DataContractSerializer LINQ to XML (XDocument)	ExtendedXmlSerializer
PHP		spatie/array-to-xml sabberworm/xmlwriter

XSL

- eXtensible Stylesheet Language
- Language that allows to represent visually an XML file
  - XML format transformation
  - Filtering and ordering
  - Value-based formatting
  - Data extracting to other formats (e.g., pdf)
- Consists of three components
  - **XSLT**. Document transformation language
  - **XPath**. Language that referentiates parts of an XML document
  - **XSL-FO**. XML document-formatting language
- XSLT Transformation online: Magictool / AppSpot

XSLT Example

- Transformation from XML to XHTML

```xml
<?xml version="1.0" encoding="ISO-8859-1"?>
<record student="Linus Torvalds">
    <subject id="1">
        <name>Basic Programming</name>
        <grade>Good</grade>
    </subject>
    ...
```

XML file

```xml
...
<tr>
    <td>Basic Programming</td>
    <td>Good</td>
</tr>
...
```

XHTML file

Browser

Academic Record

Subject	Grade
Basic Programming	Good
Operating Systems	Excellent
UX/UI	Fail

```xml
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0"
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <html>
      <body>
        <h2>Academic Record</h2>
        <table border="1">
          <tr bgcolor="#9acd32">
            <th align="left">Subject</th>
            <th align="left">Grade</th>
          </tr>
          <xsl:for-each select="record/subject">
            <tr>
              <td>
                <xsl:value-of select="name"/>
              </td>
              <td>
                <xsl:value-of select="grade"/>
              </td>
            </tr>
          </xsl:for-each>
        </table>
...
```

XSLT file

XPath

- Language that references parts of an XML document within its hierarchical tree
  - `/record` Root element
  - `/record/subject` All `subject` tags under `record`
  - `/record/subject/*` All children under `/record/subject`
  - `/record/*/name` All `name` tags that are grandchildren of `record`
  - `//*` Every element in the document
  - `/record/subject[1]` The first `subject` element under `record`
  - `/record/subject[last()]` The last `subject` element under `record`
  - `/record/subject[grade="Fail"]/name` The `name` of all `subject` tags with a grade of `Fail`
  - `/record/subject[@id="2"]` The `subject` whose attribute `id` has a value of `2`

XSL-FO

- XSL-Formatting Objects
- Styling for graphic printing
- Example. Conversion of an XML file to PDF

```xml
<?xml version="1.0" encoding="ISO-8859-1"?>
<addresses>
    <topic>My recipe addresses</topic>
    <intro>Compilation of my favourite recipe addresses.</intro>
    <address>
        <title>All Recipes</title>
        <url>https://www.allrecipes.com/</url>
        <description>America's #1 trusted recipe resource since 1997.</description>
    </address>
    <address>
        <title>Pinch of Yum</title>
        <url>https://pinchofyum.com/</url>
        <description>Simple recipes made for real, actual, everyday life.</description>
    </address>
</addresses>
```

XML file

XSL-FO

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
                xmlns:fo="http://www.w3.org/1999/XSL/Format" version="1.0">
    <xsl:template match="addresses">
        <fo:root xmlns:fo="http://www.w3.org/1999/XSL/Format">
            <fo:layout-master-set>
                <fo:simple-page-master master-name="simple" page-height="29.7cm" page-width="21cm"
                    margin-top="1cm" margin-bottom="2cm" margin-left="2.5cm" margin-right="2.5cm">
                    <fo:region-body margin-top="3cm"/>
...
```

XSLT file

```
<fo:root xmlns:fo="http://www.w3.org/1999/XSL/Format">
    <fo:layout-master-set>
    ...
            <fo:block font-size="20pt"
                      font-family="sans-serif"
                      line-height="24pt"
                      space-after.optimum="15pt"
                      text-align="center"
                      padding-top="3pt">
                My recipe addresses
            </fo:block>
    ...
```

XSL-FO file

PDF file

My recipe addresses

Compilation of my favourite recipe addresses.

All Recipes
https://www.allrecipes.com/
America's #1 trusted recipe resource since 1997.

Pinch of Yum
https://pinchofyum.com/
Simple recipes made for real, actual, everyday life.

Sources

- **Holman, G. Ken**.
  Practical Transformation Using XSLT and XPath, 14th ed.
  Crane Softwrights Ltd, 2011. ISBN: 978-1-894049-24-5.
- **Programacion.net**.
  "Convertir XML en PDF utilizando XSL-FO y FOP"
  [Converting XML in PDF using XSL-FO and FOP]
- **Sánchez Zurdo, Javier, Pablo Toharia Rabasco, and Laura Raya González**.
  Lenguajes de Marcas y Sistemas de Gestión de la Información
  [Markup Languages and Information Management Systems].
  Ra-Ma, 2011. ISBN: 978-8499641010

- Cover picture by **Markus Winkler** from Pexels